МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)
Факультет информационных технологий и прикладной математики

Кафедра вычислительной математики и программирования

Лабораторная работа №12 по курсу «Информационный поиск»

Ускорение поиска и прыжки по индексу

Студент: К. А. Арусланов
Группа: М8О-403Б-23
Преподаватель: А. А. Кухтичев
Дата: 08.11.2026
Оценка:

Москва, 2026

Содержание

1 Постановка задачи **3**

2 Метод решения **3**
2.1 Исходный поиск 3
2.2 Указатели и состояние VB-декодера 3
2.3 Булевы операции и фразы 3
2.4 Формат расширения и проверка 4

3 План и результаты тестирования **4**

4 Результаты **5**
4.1 Размер и условия эксперимента 5
4.2 Время и объём работы 5
4.3 Воспроизводимость 6

5 Вывод **7**

1 Постановка задачи

Ускорить булев и цитатный поиск по сжатому индексу, добавив переходы через части списков, которые не могут попасть в результат. Проверить сохранение выдачи, измерить время поиска, количество обработанных записей и дополнительный размер индекса. Реализовать основные структуры и алгоритмы на C++ без STL-контейнеров, подготовить автотесты, консольный запуск и веб-демонстрацию.

2 Метод решения

2.1 Исходный поиск

Используется самостоятельная сжатая копия индекса ЛР11 для 478 833 документов. Индекс содержит пространства словоформ и лемм; числовые списки хранятся как разности, записанные кодом Variable Byte. Словарь терминов использует блочное префиксное сжатие. Форматы этих данных в ЛР12 не изменялись.

При последовательном пересечении двух упорядоченных списков указатель меньшего элемента сдвигается на одну запись. Для частого слова приходится восстанавливать много ID или координат, которые заведомо меньше следующего элемента другого списка. Например, для пересечения $(2, 4, 8, 34, 35, 64, 128)$ с текущим ID 34 можно сразу перейти к 34, не читая промежуточные записи.

2.2 Указатели и состояние VB-декодера

За основу взята эвристика из лекции «Прыжки по индексу» (слайд 22): около $\sqrt{P}$ равномерных указателей для списка длины P. В реализации шаг равен $s = \lceil\sqrt{P}\rceil$, а точки сохраняются после записей $s, 2s, 3s, \ldots$. Для $P < 256$ указатели не создаются: загрузка точек короткого списка может оказаться дороже его последовательного чтения. Порог 256 выбран как простое ограничение накладных расходов и отдельно по корпусу не оптимизировался.

Каждая точка занимает 24 байта и содержит абсолютное значение, байтовое смещение следующего кодового слова внутри списка и число уже прочитанных записей. Для постинга абсолютное значение — полный 64-битный ID. Для координаты оно включает номер документа, поле и позицию токена. Из этих частей восстанавливается состояние для декодирования следующей разности. Нельзя просто перейти к произвольному байту VB-потока: нужны граница числа и предыдущее значение.

Вызов `advance(target)` ищет последнюю точку со значением не больше `target`. Переход разрешён только вперёд от текущей записи. Если значение точки равно цели, оно возвращается сразу; иначе чтение продолжается до первого значения не меньше цели. Точка больше цели не используется, поскольку перед ней может находиться нужное совпадение. При последовательных вызовах не требуется хранить сами списки в памяти: курсор загружает только указатели своего термина.

2.3 Булевы операции и фразы

В пересечении меньший операнд продвигается до текущего значения большего. Этот принцип применяется и к составным операндам. Собственные физические указатели есть только у исходных списков терминов; промежуточные множества остаются потоковыми.

Выражение A AND NOT B вычисляется как разность $A \setminus B$. Раньше сначала строилось потоковое дополнение B относительно всех документов, а затем пересекалось с A. Упрощение сохраняет смысл выражения и устраняет лишний обход таблицы документов. Для самостоятельного NOT полный обход остаётся необходимым. В OR нужно вернуть элементы обоих списков, поэтому пропускать потенциальные результаты нельзя.

В цитатном поиске координата i-го слова сдвигается на i позиций назад. После этого пересекаются сдвинутые координаты. Меньшая группа продвигается к большей с учётом обратного сдвига и границы поля. Несколько лемм одного слова объединяются по исходным координатам. Повторные слова, порядок токенов и раздельность заголовка и текста сохраняются.

2.4 Формат расширения и проверка

Четыре отдельных файла хранят таблицы терминов и точки для постингов и координат. Запись таблицы занимает 16 байт: смещение и число точек. Заголовки содержат версию, размеры, контрольные суммы FNV-64 и отпечаток заголовков сжатого индекса. Перестроение корпуса или другой вариант упаковки требует своих указателей. Незавершённое расширение имеет маркер `INCOMPLETE` и не открывается для поиска.

Команда `skip_index verify` проверяет контрольные суммы и каждую сохранённую точку против декодированного VB-потока. При обычном поиске проверяются заголовки, привязка и границы нужных списков; полная проверка выполняется отдельно после построения или переноса файлов.

3 План и результаты тестирования

1. Сравнение полных векторов ID последовательного и ускоренного поиска для AND, OR, NOT, вложенных выражений и неизвестных терминов.

2. Проверка фраз с повторными словами, нескольких лемм, порядка токенов и границы заголовка и основного текста.

3. Прямые вызовы продвижения к точному значению точки, между точками, через несколько документов и к максимальному 64-битному ID.

4. Проверка пустого индекса, повреждения, усечения, чужой привязки, незавершённой сборки, защиты существующего каталога и резерва диска.

5. Проверка CLI и передачи пути указателей из веб-формы в C++-поиск.

6. Сверка всех точек полного индекса и сравнение режимов на реальных запросах.

Синтетический корпус содержит 600 документов с разреженными 64-битными ID и длинными списками, достаточными для реальных прыжков. Помимо равенства выдачи проверяется уменьшение числа декодированных координат и посещений универсума. Пройдены все 132 проверки CTest и 283 Python-теста; 6 интеграционных проверок MongoDB пропущены. В группе ЛР12 — 9 проверок CTest и 2 Python-теста. Журнал тестов приложен в `lab12/tests.txt`. Веб-форма дополнительно проверена с настоящим индексом в обоих пространствах терминов.

4 Результаты

4.1 Размер и условия эксперимента

Расширение с таблицами занимает 200,56 МБ, то есть добавляет 14,3 % к сжатому рабочему индексу ЛР11. Здесь МБ означает 10^6 байт. Исходные VB-потоки и строки не дублируются.

Сравниваются три режима одного исполняемого файла: `linear` — последовательный алгоритм; `advance` — продвижение до цели и упрощение отрицания без указателей; `skips` — те же оптимизации с указателями. Так отдельно видна польза алгоритмических изменений и физических прыжков.

Выполнено 7 раундов для каждого пространства терминов. В каждом раунде режим запускается в новом процессе, обрабатывает один прогревочный пакет и один измеряемый. Порядок режимов циклически меняется, а порядок запросов разворачивается через раунд. Кэш ОС не очищается. Время включает разбор запроса, загрузку точек нужных терминов и перечисление всех совпадений; открытие индекса, загрузка морфологии и вывод метаданных в него не входят. В таблице приведены медианы, а не время одного запуска.

4.2 Время и объём работы

Результаты для пространства словоформ, время в мс:

Запрос	linear	advance	skips
`"god of war"`	167,14	145,85	114,01
`god AND of AND war`	10,97	8,39	9,00
`doom AND eternal`	0,90	0,98	1,05
`doom AND of`	10,10	7,25	6,88
`игры AND NOT doom`	995,97	8,68	8,79
`NOT doom`	980,02	979,22	951,30
`doom OR eternal`	0,91	0,90	0,96

Фраза «god of war» ускорилась с 167,14 до 114,01 мс, то есть в 1,47 раза. При этом относительно режима `advance` выигрыш меньше: часть ускорения даёт само продвижение до цели без повторных сравнений на каждом уровне выражения.

Число реально декодированных постингов либо координат и выполненных прыжков в тех же запросах:

Запрос	linear	skips	Прыжков
`"god of war"`	2083861	1529296	1509
`god AND of AND war`	262607	238814	576
`doom AND of`	231973	199984	462
`doom AND eternal`	10562	10198	80
`игры AND NOT doom`	156964	154101	9
`NOT doom`	7469	7469	0
`doom OR eternal`	10562	10562	0

Для «god of war» число декодированных координат сократилось примерно на 27 %. Для `игры AND NOT doom` основной выигрыш связан с упрощением выражения: посещения 478 833 записей универсума заменены разностью двух списков. Это не следует приписывать только прыжкам.

На коротком `doom AND eternal` указатели замедляют поиск: пропускается мало записей, а загрузка точек и переходы по файлам требуют времени. У одиночного термина и OR полезных прыжков нет. Самостоятельный NOT также остаётся дорогим, поскольку нужно перечислить дополнение. Небольшие колебания времени таких запросов не доказывают алгоритмического ускорения.

В обоих пространствах для всех 13 запросов и всех повторений совпали число результатов, показанные документы и контрольная сумма полного упорядоченного потока ID. Полные результаты замеров, счётчики и SHA-256 расширения, словаря и программ находятся в `lab12/summary.json` и JSONL-файлах. Счётчики отражают логическое декодирование, а не физические обращения к накопителю. Скорость холодного чтения отдельно не измерялась.

4.3 Воспроизводимость

Сборка, построение указателей, проверка, три режима CLI и запуск веб-формы описаны в `lab12_run.md`. Скрипт `run_lab12.py` проверяет точки, выполняет повторные замеры и формирует таблицы отчёта. Для построения с нуля используется новый выходной каталог и ключ `-build`. Тесты запускаются командой `python run_tests.py -lab 12`.

Исходный код: github.com/kirbasss/MAI-IR/tree/lab12.

Рис. 1 — Ссылка на ветку лабораторной работы №12

5 Вывод

В этой работе я добавил прыжки по спискам документов и координатам слов. Для фразы «god of war» удалось сократить число декодированных координат и ускорить поиск примерно в полтора раза относительно последовательного варианта. Результаты поиска при этом сохранились.

Самый большой выигрыш на запросе с AND NOT получился благодаря устранению лишнего обхода всего корпуса. При этом указатели помогают не везде: короткие списки иногда выгоднее читать подряд. За ускорение пришлось заплатить дополнительными 200,56 МБ на диске. Получился полезный выигрыш для длинных пересечений, но универсального ускорения всех запросов нет.